Forward Kinematics Kernel for Improved Proxy Collision Checking

Nikhil Das and Michael C. Yip

***Abstract*—Kernel functions may be used in robotics for comparing different poses of a robot, such as in collision checking, inverse kinematics, and motion planning. These comparisons provide distance metrics often based on joint space measurements and are performed hundreds or thousands of times a second, continuously for changing environments. We introduce a new kernel function based on forward kinematics (FK) to compare robot manipulator configurations. We integrate our new FK kernel into our proxy collision checker, Fastron, that previously showed significant speed improvements to collision checking and motion planning. With the new FK kernel, we realize a two-fold speedup in proxy collision check speed, 8 times less memory, and a boost in classification accuracy from 74% to over 95% for a 7 degrees-of-freedom robot arm compared to the previously-used radial basis function kernel. Compared to state-of-the-art geometric collision checkers, with the FK kernel, collision checks are now 9 times faster. To show the broadness of the approach, we apply Fastron FK in OMPL across a wide variety of motion planners, showing unanimously faster robot planning.**

***Index Terms*—Collision avoidance, kinematics, motion and path planning.**

I. Introduction

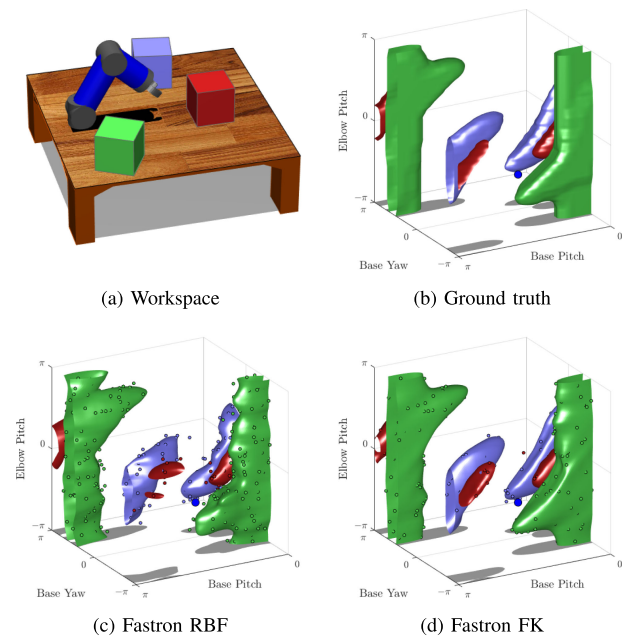
MOTION planning for robotics is the task of finding a sequence of feasible robot states between a start and goal state. Feasible robot states are those that satisfy problem-specific constraints, generally one of which is to remain out of collision from objects in the environment. Motion planning for a robot is often performed in its configuration space (C-space), a space in which each element represents a unique configuration of the robot. Each configuration in the robot's C-space can belong to one of two regions: $\mathcal{C}_{free}$ or $\mathcal{C}_{obs}$. A configuration is in the collision-free $\mathcal{C}_{free}$ region if the robot is not in contact with any workspace obstacle when in that corresponding configuration; otherwise, the configuration is in the in-collision $\mathcal{C}_{obs}$ region [1], [2]. For robot manipulators with many controllable degrees-of-freedom (DOF), the dimensionality of the C-space may be large as each joint contributes to defining the robot's configuration.

Manuscript received September 10, 2019; accepted January 8, 2020. Date of publication January 31, 2020; date of current version February 19, 2020. This letter was recommended for publication by Associate Editor N. Amato and A. Faust upon evaluation of the reviewers' comments. *(Corresponding author: Nikhil Das.)*

The authors are with the Advanced Robotics and Controls Laboratory, University of California, San Diego, CA 92093 USA (e-mail: nrdas@ucsd.edu; yip@uscd.edu).

This article has supplementary downloadable material available at https://ieeexplore.ieee.org, provided by the authors.

Digital Object Identifier 10.1109/LRA.2020.2970645



(a) Workspace (b) Ground truth (c) Fastron RBF (d) Fastron FK

Fig. 1. (a) A 3 DOF robot and multiple cube obstacles. (b) The ground truth C-space model determined by calling a geometric collision detector on a grid of samples. (c) A Fastron model using the RBF kernel from prior work. (d) A Fastron model using the proposed FK kernel. Both versions of the Fastron models are trained on the same data, and the resulting support points for each model are shown. The Fastron FK model closely matches the ground truth model while the Fastron RBF model is uneven and requires many more support points.

An example workspace and its corresponding C-space for a 3 DOF robot arm are shown in Fig. 1a–1b.

Determining feasibility of a robot state usually involves discrete collision checking to determine whether the query robot state would intersect with an environmental obstacle [3]. Repeated collision checking is computationally expensive and takes up to 90% of the computation time during motion planning [4]. An open challenge exists for real-time, accurate, dynamic robot planning in changing environments. To address this challenge, we seek to create a machine learning-based C-space model to be used as a faster proxy to computationally-burdensome geometric collision checkers upon which most motion planners rely.

A. Contributions

In this letter, we present a new kernel function, or similarity function, for improved learning-based C-space modeling for robot manipulators. We show improved proxy

2377-3766 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.

collision checking performance in terms of both collision check times and accuracy with the new kernel, and we demonstrate the kernel's usefulness in accelerating motion planning.

In previous work, we described the Fastron algorithm, which creates a nonparametric model inspired by the kernel perceptron algorithm and efficiently checks for changes in the C-space due to moving workspace obstacles [5]. The input into the Fastron model is a robot configuration, and the output is its predicted collision status. In previous work, a radial basis function (RBF) was used for the kernel function, which isotropically compares the joint values of a robot manipulator. While the RBF kernel is an adequate general-purpose kernel function, it fails to capture the influence each joint has in the manipulator's workspace. In this work, we replace the RBF kernel function with a new kernel function we call the forward kinematics (FK) kernel. The benefits of replacing the RBF kernel with the FK kernel may be observed in Fig. 1c–1d, where the Fastron model closely resembles the ground truth with the FK kernel but is uneven and requires many more points to represent the model when using the original RBF kernel.

We show that the FK kernel is positive definite and prove our modeling algorithm can always find a set of weights that correctly predicts the collision status of a training set. We also show that when using the Fastron model as a proxy collision checker, using the FK kernel can provide collision status predictions twice as fast as when using the RBF kernel and requires approximately an order of magnitude fewer support points to represent the model. Using the FK kernel improves the accuracy of collision status prediction to be over 95% for a 7 DOF robot arm, a significant improvement over the 74% accuracy achieved with the RBF kernel. We apply the Fastron algorithm with the FK kernel to various motion planners, demonstrating up to 3 times faster motion plans compared to when using other collision checkers.

B. Related Work

The following works describe proxy checking methods that try to avoid computationally-burdensome collision checking and methods that utilize control points (which are used in the proposed FK kernel) for other kinematic contexts.

1) Configuration Space Modeling: Various machine learning-based techniques have been developed to model C-space or for proxy collision checking, including support vector machine models (SVMs) [6], Gaussian mixture models (GMMs) [7], k-nearest neighbor models (KNNs) [8], and kernel perceptrons [5]. Most of these methods provide either high accuracy but very poor computational speed, or high computational speed but poor accuracy and limited capacity to model changing environments.

Pan *et al.*'s SVM method learns a C-space model for a pair of rigid-body objects [6]. Their method provides near-perfect models because they employ an active learning strategy that iteratively improves the precision of the model, and they use their SVM-based model for distance/penetration depth estimation between two bodies using a constrained nonlinear optimizer. However, to apply this method to a kinematic chain, a new model must be learned offline for each pair of objects, including each link in the kinematic chain and each workspace obstacle.

In [7], active learning of a GMM model is associated with sample-based motion planners itself by using the samples from a rapidly-exploring random tree (RRT) motion planner to modify the GMM. GMMs are constrained by the fixed dimensionality of the model.

KNNs have also been used for faster proxy collision detection for static environments [8]. Locality-sensitive hashing is utilized for extremely fast query times. However, the problem of rehashing given changing environments makes this unsuitable for changing environments.

Our Fastron algorithm uses a kernel perceptron model to globally represent the C-space and uses an active learning strategy to adapt to a changing workspace. It was shown that collision checks may be an order of magnitude faster than state-of-the-art collision detectors [5]. However, the accuracy of the model must be improved to reduce the search time for valid configurations and to reduce the amount of time spent repairing infeasible parts of a motion plan.

While each of these learning-based methods generates an approximate C-space model, the workspace and C-space are treated as completely separate entities and the link that relates the two spaces is more or less neglected. In this work, we aim to link the two spaces via FK.

2) Control Points for Collision-Free Motion: Control points are a set of hand-selected points placed on a robot and their locations may be used to indicate the position of a rigid-body robot or a kinematic chain manipulator. Control points should be chosen to constrain the configuration of the robot, and the minimum number of these points depends on the number of DOF of the robot [1]. FK provides the positions of these control points for manipulators given a set of joint values, and we consider FK to be the link that relates a robot's position in C-space to its position in the workspace.

Artificial potential fields can be used for motion planning in a robot's C-space, where the sum of attractive and repulsive forces drive the robot away from C-space obstacles and toward a goal [9]. Potential fields can be applied to kinematic chain manipulators using control points placed along the kinematic chain [1].

RelaxedIK, a motion-synthesis method for manipulators, uses a neural network to estimate the risk of self-collision for an input robot configuration [10]. Control points are placed at each joint and their locations in the workspace are included in the input to the network. The use of control points allows estimation of the self-collision costs two orders of magnitude faster than a distance-based method, and optimizing over these self-collision costs allows finding configurations that are free from self-collisions.

In this work, we utilize control points in creating a similarity function between manipulator configurations, and we leverage how FK relates a manipulator's C-space to its workspace for more accurate proxy collision checking.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.

II. Background

A. Forward Kinematics

FK for a kinematic chain determines the pose (position and orientation) of a point on the chain given the joint values. When using the standard Denavit-Hartenberg (D-H) convention, the homogenous transformation matrix of the pose of joint i in the frame of joint $i-1$ is

$$\boldsymbol{^{i-1}A_i} = \begin{bmatrix} \mathrm{c}(\theta_i) & -\mathrm{c}(\alpha_i)\mathrm{s}(\theta_i) & \mathrm{s}(\alpha_i)\mathrm{s}(\theta_i) & a_i\mathrm{c}(\theta_i) \\ \mathrm{s}(\theta_i) & \mathrm{c}(\alpha_i)\mathrm{c}(\theta_i) & -\mathrm{s}(\alpha_i)\mathrm{c}(\theta_i) & a_i\mathrm{s}(\theta_i) \\ 0 & \mathrm{s}(\alpha_i) & \mathrm{c}(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

where d_i and a_i are the distances between the joints' origins along the ith x-axis and z-axis, respectively, θ_i and α_i are the angles between the joints' x-axes measured along the ith z-axis and vice versa, respectively, and $\mathrm{c}(\cdot)$ and $\mathrm{s}(\cdot)$ represent $\cos(\cdot)$ and $\sin(\cdot)$, respectively. θ_i and d_i are the joint variables for revolute and prismatic joints, respectively, and a configuration of a robot with D DOF is represented as a D-dimensional vector of these joint variables. The other D-H parameters are fixed depending on the geometry of the robot. Note that when using the standard D-H convention, the origin of the frame of the ith joint is placed at the distal end of the ith link. For more details, see [11].

Starting with the base transform relative to a world frame $\boldsymbol{^wA_0}$, the pose of the ith joint is $\boldsymbol{^wA_i} = \boldsymbol{^wA_0\,^0A_1 \ldots\,^{i-1}A_i}$, where each transform in the chain is dependent on the robot configuration as described in the previous paragraph. The pose for an arbitrary point on the ith link is found by appending an additional static transformation $\boldsymbol{T}$ to the pose of the ith joint: $\boldsymbol{^wA_iT}$.

Let $\mathrm{pos}(\cdot)$ extract the position from a homogeneous transformation matrix, i.e., the first three elements of the last column. We can thus represent the position of the mth control point for configuration $\boldsymbol{x} \in \mathbb{R}^D$ as

$$FK_m(\boldsymbol{x}) = \mathrm{pos}\left(\boldsymbol{^wA_{j_m}T_m}\right) \quad (2)$$

where $\boldsymbol{T_m}$ is the static transform applied to the the pose of the joint with index j_m.

B. Binary Classification and Kernel Functions

A binary classifier predicts the class label of a query point. A binary classifier model is fit to a dataset $\mathcal{X} = \{\mathcal{X}_1, \ldots, \mathcal{X}_N\}$ and its associated vector of labels $\boldsymbol{y}$. Each $\mathcal{X}_i \in \mathbb{R}^D$ represents a configuration of a robot. We let $\boldsymbol{y}_i = +1$ denote $\mathcal{X}_i \in \mathcal{C}_{obs}$ and $\boldsymbol{y}_i = -1$ denote $\mathcal{X}_i \in \mathcal{C}_{free}$.

Binary classification models often use kernels to describe nonlinear seperation of classes. A kernel function $K : \mathbb{R}^D \times \mathbb{R}^D \to \mathbb{R}$ is a similarity function that should provide a large score for similar configurations and a low score for dissimilar configurations. RBF kernels, which are functions of the distance between two points, such as the Gaussian kernel, are popular. In this letter, we use the second-order rational quadratic kernel [12] for all RBF kernels, defined as

$$K_{RQ}(\boldsymbol{x}, \boldsymbol{x}') = \left(1 + \frac{\gamma}{2}\|\boldsymbol{x} - \boldsymbol{x}'\|^2\right)^{-2} \quad (3)$$

where $\gamma > 0$ is a parameter dictating the width of the kernel.

Each element of a kernel matrix $\boldsymbol{K}$ is defined elementwise for each pair of points in $\mathcal{X}$: $\boldsymbol{K}_{ij} = K(\mathcal{X}_i, \mathcal{X}_j)$. $\boldsymbol{K}$ is called positive definite (PD) if $\boldsymbol{\alpha}^\mathsf{T}\boldsymbol{K}\boldsymbol{\alpha} \geq 0\, \forall \boldsymbol{\alpha} \in \mathbb{R}^N$, and equality occurs only for $\boldsymbol{\alpha} = \boldsymbol{0}$. $\boldsymbol{K}$ is considered to be positive semidefinite (PSD) if equality may occur for $\boldsymbol{\alpha} \neq \boldsymbol{0}$.

A PD kernel is one that yields a PD kernel matrix $\boldsymbol{K}$ when each configuration in set $\mathcal{X}$ is unique [13]. Positive definiteness is an important property for a kernel function because it guarantees nonsingularity for the kernel matrix and represents an implicit mapping to a higher dimensional feature space according to Mercer's theorem [14]. PSD kernels and matrices do not have the same guarantees their PD counterparts have. $K_{RQ}(\cdot, \cdot)$ is an example of a PD kernel [12]. Summing PD kernels or multiplying PD kernels with a scalar also yields a PD kernel [13].

III. Methods

A. Forward Kinematics Kernel

1) Definition: The major insight of this letter is in a more intuitive distance metric by computing distances in the workspace rather than in the joint space. Computing distances directly between joint configurations does not take into consideration the robot's position in the workspace. Consider the blue two-link arm shown in Fig. 2a. Comparing the pink and green configurations to the blue, the pink configuration appears much closer to the blue than the green does. However, the distance in the C-space between the blue and pink configurations (represented as colored dots) is equal to that of the blue and green configurations as can be seen in Fig. 2b and 2c. Consequently, according to the RBF kernel, the blue configuration is as similar to the pink configuration as it is the green configuration as shown in Fig. 2b.

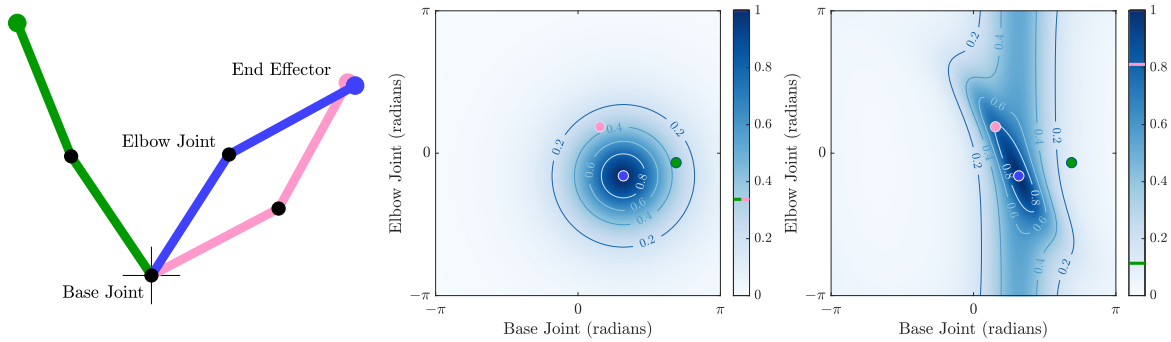
Rather than comparing the joint values of two configurations directly, we use FK and compare distances in the workspace between control points on the arm. We can thus define the FK kernel as a sum of RBF kernels evaluated between each corresponding control point location:

$$K_{FK}(\boldsymbol{x}, \boldsymbol{x}') = \frac{1}{M}\sum_{m=1}^{M} K_{RQ}\left(FK_m(\boldsymbol{x}), FK_m(\boldsymbol{x}')\right) \quad (4)$$

where $FK_m(\cdot)$ provides the position of the mth control point and M is the number of control points representing the kinematic chain.

In this letter, we only place control points at joint frame origins (using the standard D-H convention) by setting $\boldsymbol{T_m} = \boldsymbol{I}$ for each control point. In other words, a control point is placed at the distal end of each link. However, control points may be placed at arbitrary locations along the manipulator if desired, as long as the control points constrain the positions of the links. If the origins of the frames of two subsequent joints in the kinematic chain are coincident, we only place one control point at the overlapping origin. In other words, using the standard D-H convention, if a revolute joint has both d_i and a_i equal

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.



(a) Example workspace (b) RBF kernel in configuration space (c) FK kernel in configuration space

Fig. 2. (a) A workspace representation of one configuration of a two-link arm shown in blue along with two alternate configurations of the same robot shown in green and pink. (b) A visualization of the RBF kernel in C-space centered on the blue configuration. According to the RBF kernel, the blue configuration is as similar to the pink configuration as it is the green configuration, as they reside on the same isocontour. (c) A visualization of the FK kernel in C-space centered on the blue configuration. According to the FK kernel, the blue configuration is more similar to the pink configuration than to the green, which is more representative of the proximities of the blue and pink configurations in the workspace representation.

to 0, a control point is not placed for this joint as its origin overlaps with the preceding joint origin. Ultimately, one control point is placed for each link, which also matches the suggested number of control points for the potential fields method for manipulators [1].

Fig. 2c provides a visualization of the FK kernel for the blue two-link arm in Fig. 2a, where control points are placed at the location labeled "Elbow Joint" and "End Effector". According to the FK kernel, the similarity score between the blue and pink configurations is about 0.8, while the similarity between the blue and green is about 0.1. The similarity scores produced by the FK kernel better represent the differences we perceive in the positions of the robots in the workspace.

2) Positive Definiteness: As previously mentioned, a PD kernel guarantees a PD and nonsingular kernel matrix and represents an implicit mapping to a feature space. The RBF kernel defined in Eq. 3 evaluated on dataset $\mathcal{X}$ will give rise to a PD kernel matrix as long as each configuration in $\mathcal{X}$ is unique. We show below this is also true for the FK kernel.

The FK kernel matrix based on the FK kernel defined in Eq. 4 is a sum of RBF kernel matrices evaluated on M control points: $\boldsymbol{K} = \frac{1}{M}\sum_i^M \boldsymbol{K_m}$. We henceforth refer to the RBF kernel matrices that comprise the FK kernel matrix as *summand kernel matrices*. The mth summand kernel matrix $\boldsymbol{K_m}$ is PD as long as the locations of the mth control point (determined by applying $FK_m(\cdot)$ to the configurations in $\mathcal{X}$) are unique. However, depending on the choices of control points or kinematic redundancy, it is possible for two unique configurations in $\mathcal{X}$ to yield the same control point location. For example, Fig. 3 shows four unique configurations of a two-link arm, but there are only three unique locations for each control point (represented as a hollow square and circle on each configuration). With coincident control point locations, the summand kernel matrices are demoted to PSD.

Despite possibly having PSD summand kernels, the *FK kernel matrix is PD* if the intersection of the nullspaces of the summand matrices is the zero vector.

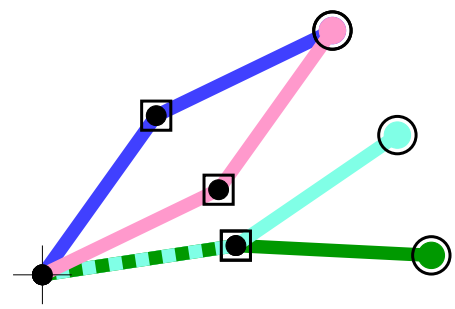


Fig. 3. Four unique configurations of a two-link arm. There are two control points selected along the arm, marked by the square and hollow circle. Overlapping links are marked with a dashed line. Despite being unique configuations, some control points overlap, causing summand kernel matrices to be positive semidefinite. However, the FK kernel is positive definite as long as the configurations with coincident control points are distinguishable based on the other control points.

Claim 1: The FK kernel matrix $\boldsymbol{K} = \frac{1}{M}\sum_{m=1}^M \boldsymbol{K_m}$ is PD if the nullspaces of each summand kernel matrix $\boldsymbol{K_m}$ share only the zero vector, $\bigcap_{m=1}^M N(\boldsymbol{K_m}) = \boldsymbol{0}$.

Proof: Assume $\boldsymbol{v} \in \bigcap_{m=1}^M N(\boldsymbol{K_m})$. Thus, $\boldsymbol{K_1 v} + \cdots + \boldsymbol{K_M v} = M\boldsymbol{K v} = 0 \Rightarrow \bigcap_{m=1}^M N(\boldsymbol{K_m}) \subseteq N(\boldsymbol{K})$.

For $\boldsymbol{K}$ to be PD, it must also be nonsingular, which means $N(\boldsymbol{K}) = \boldsymbol{0}$. Thus, $\bigcap_{m=1}^M N(\boldsymbol{K_m}) = \boldsymbol{0}$. ■

What Claim 1 means intuitively is that even if some control point locations coincide for two unique configurations, the FK kernel is PD if these configurations are distinguishable using the locations of the other control points. For example, in Fig. 3, the locations of the control points marked with hollow circles differentiate the configurations with coincident control points marked with squares, and vice versa. Note that in practice, if the configurations in $\mathcal{X}$ are generated via random sampling, control point locations should almost surely be unique.

B. Fastron Model and Algorithm

The Fastron algorithm (described in detail in [5]) generates a binary classification model that is used for proxy collision

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.

checking. The Fastron model is a weighted sum of kernel functions: $f(\boldsymbol{x}) = \sum_{i=1}^{N} K(\mathcal{X}_i, \boldsymbol{x})\boldsymbol{\alpha}_i$, where $\boldsymbol{\alpha} \in \mathbb{R}^N$ is a vector of model weights, $\mathcal{X}$ is a set of N configurations, and $K(\cdot,\cdot)$ is a PD kernel. Proxy collision detection is performed by determining the sign of $f(\boldsymbol{x})$, where $f(\boldsymbol{x}) > 0$ denotes the query configuration $\boldsymbol{x}$ is predicted to be in $\mathcal{C}_{obs}$ and $f(\boldsymbol{x}) < 0$ denotes $\mathcal{C}_{free}$. $\boldsymbol{x}$ is considered to be correctly classified if its margin $m(\boldsymbol{x}) = yf(\boldsymbol{x})$ is positive, where y is its true label.

The weight vector $\boldsymbol{\alpha}$ is found by minimizing the loss

$$\mathcal{L}(\boldsymbol{\alpha}) = \frac{1}{2}\boldsymbol{\alpha}^{\mathsf{T}}\boldsymbol{K}\boldsymbol{\alpha} - \boldsymbol{y}^{\mathsf{T}}\boldsymbol{B}\boldsymbol{\alpha} \tag{5}$$

subject to $\boldsymbol{y}_i\boldsymbol{\alpha}_i \geq 0\ \forall i$, where $\boldsymbol{y}$ is a vector of labels for dataset $\mathcal{X}$, $\boldsymbol{K}$ is the kernel matrix for $\mathcal{X}$, and $\boldsymbol{B}$ is a diagonal matrix where the ith diagonal element is $b_i = \beta^{0.5(\boldsymbol{y}_i+1)}$ and $\beta \geq 1$ is a user-selected conditional bias parameter, which biases the model toward predicting $\mathcal{C}_{obs}$ for more conservative predictions. This loss function is similar to that used in SVMs [15]. The Fastron algorithm utilizes greedy coordinate descent and exits as soon as the margin $m(\mathcal{X}_i) = \boldsymbol{y}_i\boldsymbol{F}_i > 0\ \forall \mathcal{X}_i \in \mathcal{X}$, where $\boldsymbol{F}_i = f(\mathcal{X}_i)$. Some advantages of greedy coordinate descent we realize are it promotes model sparsity (and thus computational efficiency), allows us to perform lazy kernel matrix evaluation (i.e., a column of $\boldsymbol{K}$ can be computed and stored only when needed), and allows us to define a cheap update rule (described below) [5].

In what follows, parenthetical superscripts denote the training iteration upon which the given value depends. On iteration n, $\boldsymbol{\alpha}^{(n+1)}$ and $\boldsymbol{F}^{(n+1)}$ are determined incrementally:

$$\Delta\boldsymbol{\alpha}_i^{(n)} = b_i\boldsymbol{y}_i - \boldsymbol{F}_i^{(n)} \tag{6}$$

$$\boldsymbol{\alpha}^{(n+1)} = \boldsymbol{\alpha}^{(n)} + \Delta\boldsymbol{\alpha}_i^{(n)}\boldsymbol{e}(\boldsymbol{i}) \tag{7}$$

$$\boldsymbol{F}^{(n+1)} = \boldsymbol{F}^{(n)} + \Delta\boldsymbol{\alpha}_i^{(n)}\boldsymbol{K}\boldsymbol{e}(\boldsymbol{i}) \tag{8}$$

where $\boldsymbol{e}(\boldsymbol{i})$ is the ith standard basis vector. The descent direction, as stated previously, is determined greedily by selecting the point with the most negative margin:

$$i = \underset{j}{\operatorname{argmin}}\left(m^{(n)}(\mathcal{X}_j)\right) \tag{9}$$

This descent rule guarantees positive margin for all points in $\mathcal{X}$ in a finite number of iterations.

Claim 2: Minimization of $\mathcal{L}(\boldsymbol{\alpha})$ with the greedy coordinate descent rule defined in Eqs. 7 and 9 will always eventually yield a model with positive margin for all samples given nonsingular kernel matrix $\boldsymbol{K}$.

Proof: If $\min_i m^{(n)}(\mathcal{X}_i) \leq 0$, an upper bound on the change in loss per descent step is $\sup(\mathcal{L}(\boldsymbol{\alpha}^{(n+1)}) - \mathcal{L}(\boldsymbol{\alpha}^{(n)})) = \sup(-\frac{1}{2}(b_i - m^{(n)}(\mathcal{X}_i))^2) = -\frac{1}{2}$. A lower bound of $\mathcal{L}(\boldsymbol{\alpha})$ is $\inf \mathcal{L}(\boldsymbol{\alpha}) = \mathcal{L}(\boldsymbol{K}^{-1}\boldsymbol{B}\boldsymbol{y}) = -\frac{1}{2}\boldsymbol{y}^{\mathsf{T}}\boldsymbol{B}\boldsymbol{K}^{-1}\boldsymbol{B}\boldsymbol{y}$ for nonsingular $\boldsymbol{K}$. A loose upper bound on the number of descent steps required to reach $\mathcal{L}(\boldsymbol{K}^{-1}\boldsymbol{B}\boldsymbol{y})$ from initial loss $\mathcal{L}(\boldsymbol{\alpha}^{(0)})$ is thus $\frac{\mathcal{L}(\boldsymbol{K}^{-1}\boldsymbol{B}\boldsymbol{y}) - \mathcal{L}(\boldsymbol{\alpha}^{(0)})}{-\frac{1}{2}} = \boldsymbol{y}^{\mathsf{T}}\boldsymbol{B}\boldsymbol{K}^{-1}\boldsymbol{B}\boldsymbol{y} + 2\mathcal{L}(\boldsymbol{\alpha}^{(0)})$. The margin is b_i for $\mathcal{X}_i$ when $\boldsymbol{\alpha} = \boldsymbol{K}^{-1}\boldsymbol{B}\boldsymbol{y}$, which is positive because $b_i \geq 1$.

If $\min_i m^{(n)}(\mathcal{X}_i) > 0$, the model at iteration n successfully provides a positive margin for all samples. ■

Algorithm 1: Fastron Model Update Algorithm.

Input: Dataset of configurations $\mathcal{X}$, collision status labels $\boldsymbol{y}$
Output: Updated $\boldsymbol{\alpha}$, support set of configurations $\mathcal{S}$

```
// Get values from previous update
α, F, K ← loadPreviousModel()
for iter = 1 to iter_max do
    // Check for misclassifications
    if min yF ≤ 0 then
        i ← argmin yF
        computeKernelMatrixColumn(i)
        // Add/adjust support point
        if count(α ≠ 0) < S_max OR α_i ≠ 0 then
            Δα ← b_i y_i − F_i
            α_i ← α_i + Δα
            F ← F + Δα K e(i)
            continue
    // Remove redundant support points
    if max y(F − α) > 0 subject to α ≠ 0 then
        i ← argmax y(F − α) subject to α ≠ 0
        F ← F − α_i K e(i)
        α_i ← 0
        continue
    break
// Remove all elements corresponding to
   non-support points
α, S, F, K ← removeNonsupportPoints()
return α, S
```

Claim 2 means the weight update algorithm can terminate once all training samples have positive margin or will otherwise work toward achieving positive margin for all samples. Claim 2 is contigent on $\boldsymbol{K}$ being nonsingular, which is shown to be the case for the FK kernel in Claim 1.

After finding a model with positive margin for all training points, points in the training set with nonzero value in $\boldsymbol{\alpha}$ comprise the support set $\mathcal{S} \subseteq \mathcal{X}$. To promote model sparsity, support points are removed from $\mathcal{S}$ (by setting their corresponding weight to 0) if they would have positive margin even after their removal from $\mathcal{S}$.

The Fastron model update algorithm is summarized in Algorithm 1. An iteration limit $iter_{max}$ and a limit on the size of the support set $\mathcal{S}_{max}$ can be set to limit update times and model sizes at the cost of model accuracy. $\mathrm{loadPreviousModel}()$ initializes the values in each data structure to 0 or loads a previously-trained model if it exists, $\mathrm{computeKernelMatrixColumn}(i)$ computes the ith column of $\boldsymbol{K}$, and $\mathrm{removeNonsupportPoints}()$ removes elements from any data structure corresponding to a non-support point because these points are not needed for classification.

After model updating, a two-stage active learning strategy searches for changes in the C-space in case workspace obstacles move. In the first stage, random samples are generated near each support point to search for small perturbations to the C-space obstacles. In the second stage, uniformly random samples are generated in the C-space to search for new obstacles entering the workspace of the robot. A geometric collision check is performed for each support point and new sample generated via active learning to update $\boldsymbol{y}$. With the updated training set, the model update procedure is repeated.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.

IV. Results

In this section, we evaluate the performance of the FK kernel by examining how it improves proxy collision checking and motion planning. We use the 7 DOF arms of the Baxter robot for these experiments. We use Fastron with the proposed FK kernel and with the original RBF kernel, denoted as Fastron FK and Fastron RBF, respectively. Via grid search, we select $\gamma = 100$ for Fastron FK and $\gamma = 10$ for Fastron RBF and $\beta = 100$ for both models, and both models are initialized with 5000 training samples. We select the maximum number of support points and maximum number of iterations for model updates to be large enough to allow full convergence. The geometric collision detectors compared are GJK [16] and FCL [17]. GJK can determine whether two convex shapes are intersecting, and FCL is the collision detection library used in the MoveIt! motion planning framework [18]. We use GJK as a faster, approximate geometric method and use FCL as the ground truth collision method.

A. Collision Checking

To compare collision checking performance, we compare model correctness, query times, model sizes, and model update times. Measures of model correctness include the overall classification accuracy, true positive rate (TPR), and true negative rate (TNR), defined as

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + TN + FP} \tag{10}$$

$$\text{TPR} = \frac{TP}{TP + FN} \tag{11}$$

$$\text{TNR} = \frac{TN}{TN + FP} \tag{12}$$

where TP and FP represent the number of samples in the test set that are correctly and incorrectly predicted to be in $\mathcal{C}_{obs}$, respectively, and TN and FN represent the same but for the $\mathcal{C}_{free}$ class. TPR and TNR are useful measures of correctness when the volumes of $\mathcal{C}_{obs}$ and $\mathcal{C}_{free}$ are unbalanced. Model size $|\mathcal{S}|$ is the count of support points required to represent the model for the proxy collision check methods. The results of the experiments are averaged over 200 trials, where each trial uses one randomly-sized obstacle that moves up to 30 times in a random direction, and the models are updated between each obstacle movement.

The query timings, model sizes, and model update timings are tabulated in Table I. Query timings refer to the time per collision check, model sizes refer to the number of support points required for the Fastron models, and model update timings refer to the time required to update a trained Fastron model when obstacles move. The initial model training (i.e., labeling 5000 random configurations using FCL and training the model starting from $\boldsymbol{\alpha} = 0$) took on average 133.9 ms and 164.6 ms for Fastron FK and Fastron RBF, respectively. The majority of this initial training time (approximately 115 ms) for both methods is dedicated toward labeling the dataset using FCL. While Fastron FK has 7.6 times fewer support points than Fastron RBF, the FK kernel is more expensive to compute than the RBF kernel, causing Fastron FK to be only 1.8 times faster than Fastron RBF for query times. Fastron FK beats GJK and FCL by a factor of 5.6 and 8.8, respectively, for query times. Since there are fewer support points in the Fastron FK model, the update time (largely throttled by labeling new data via FCL) is about 2.4 times faster for Fastron FK than for Fastron RBF.

The correctness of the approximate collision methods are shown in Fig. 4, illustrating the advantages of Fastron FK over Fastron RBF. The accuracy, TPR, and TNR of Fastron FK are much higher than those of Fastron RBF. In fact, Fastron FK performs similarly in terms of correctness to GJK, but is 5.6 times faster. Fastron FK achieves an accuracy of 96.1%, a significant improvement over Fastron RBF's accuracy of 73.7%. While the accuracy, TPR, and TNR are roughly balanced for Fastron FK, Fastron RBF's TPR is much higher than its accuracy and TNR because it prioritizes the $\mathcal{C}_{obs}$ label over the $\mathcal{C}_{free}$ label. While biasing toward the $\mathcal{C}_{obs}$ label yields a more conservative

TABLE I
Performance of Fastron FK and Fastron RBF Compared Against Geometric Collision Detection Methods (GJK and FCL). Lower Is Better for Query Time, Model Size $|\mathcal{S}|$, and Model Update Time

Method	Query Time	$\lvert\mathcal{S}\rvert$	Update Time
Fastron FK	$2.6 \pm 0.5\ \mu s$	329.7 ± 120.6	$56.8 \pm 5.5\ ms$
Fastron RBF	$4.8 \pm 1.2\ \mu s$	2505.2 ± 616.7	$134.5 \pm 32.9\ ms$
GJK	$14.5 \pm 0.1\ \mu s$	—	—
FCL	$22.8 \pm 0.3\ \mu s$	—	—

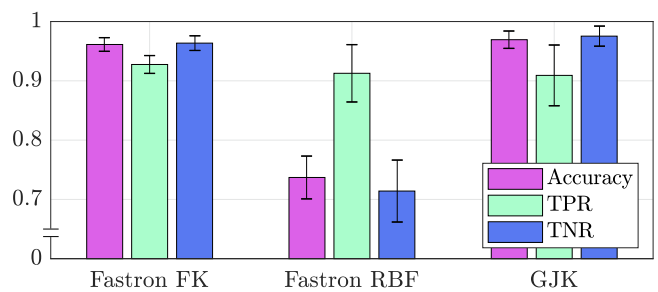
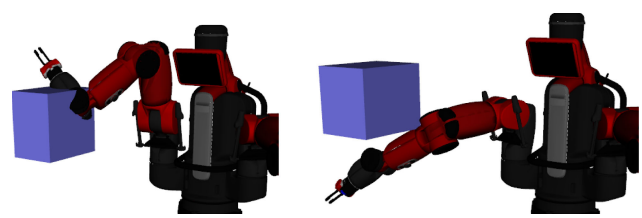


Fig. 4. A comparison of the correctness of Fastron FK, Fastron RBF [5], and GJK [16] with a single, randomly-sized moving obstacle. Fastron FK has much better accuracy, TPR, and TNR than Fastron RBF and performs similarly to GJK.



(a) Fastron RBF (b) Fastron FK

Fig. 5. Typical failure cases using the RBF kernel and the FK kernel, where the configuration is mistakenly labeled as $\mathcal{C}_{free}$ but is truly $\mathcal{C}_{obs}$. With the RBF kernel, the robot blatantly intersects with the workspace obstacle. With the FK kernel, the robot slightly intersects with the obstacle.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.

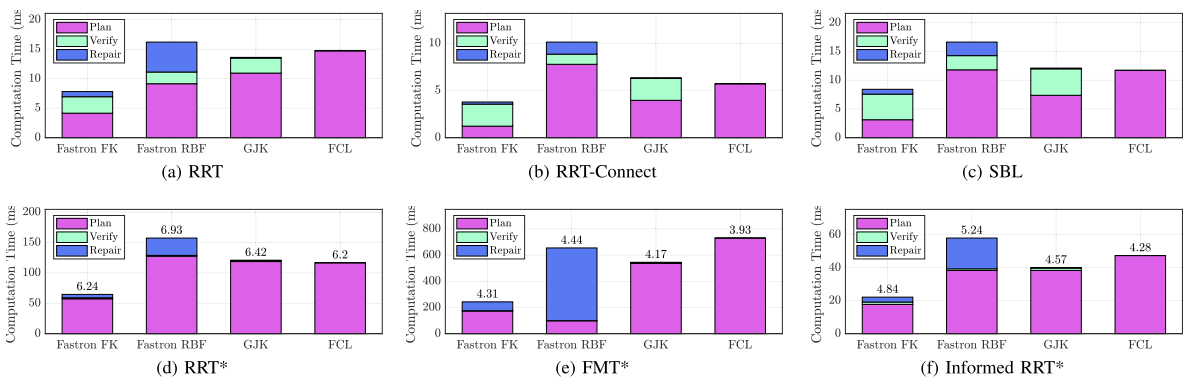


Fig. 6. Motion planning results using various collision detection methods: Fastron with FK kernel, Fastron with RBF kernel, GJK, and FCL (which serves as the ground truth collision detector). The obstacle in the workspace is repeatedly moved. For each approximated collision detection method, the path is validated and fixed using FCL in the Verify and Repair steps. The numbers above each optimizing planner's bars are the average lengths of the initial motion plans.

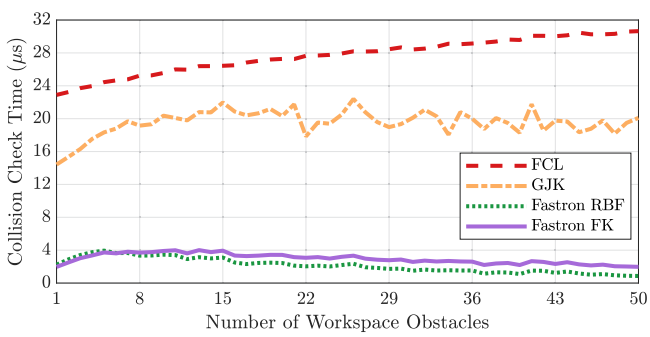


Fig. 7. Query times for Fastron FK, Fastron RBF [5], GJK [16], and FCL [17] against the number of workspace obstacles. Both GJK and FCL increase in query times as the number of obstacles increase, while both Fastron methods increase slightly before decreasing because the number of support points required to represent the space are the most when the class sizes are more balanced.

model compared to not biasing toward $\mathcal{C}_{obs}$, it also reduces the volume of the valid region of C-space and increases search time during sampling-based motion planning, which we will see is the case for Fastron RBF in the next section.

Fig. 5 shows typical false negatives when using Fastron FK and Fastron RBF, where the true collision status is $\mathcal{C}_{obs}$ but the Fastron models incorrectly predict $\mathcal{C}_{free}$. Using the RBF kernel, the robot significantly intersects with the workspace obstacle. The false negatives have an average (over 500 random environments) penetration depth of 3.3 cm, and 22.1% of them have more than 5 cm of penetration. On the other hand, using the FK kernel, the robot only slightly intersects with the workspace obstacles. The false negatives have an average penetration depth of 1.6 cm, and 4.1% of them have a penetration greater than 5 cm. A lower penetration depth for false negatives shows that Fastron FK has less severe mistakes than Fastron RBF in the workspace.

In an additional experiment, we increase the number of obstacles to see the effect on query times. Fig. 7 shows the query times for each method with respect to the number of workspace obstacles for up to 50 randomly-placed obstacles. We can see that the geometry-based methods, GJK and FCL, increase in query times as the number of obstacles increase. On the other hand, Fastron-based query times first increase and then decrease because fewer support points are required when one collision status is more prevalent than the other. Fewer support points are required by both Fastron FK and Fastron RBF when $\mathcal{C}_{obs}$ dominates the C-space, and because each FK kernel (Eq. 4) evaluation is a sum of RBF kernels (Eq. 3), Fastron RBF inevitably becomes faster than Fastron FK when both models require fewer support points to represent the C-space. Nonetheless, the accuracy of Fastron FK is consistently high (between 94–98%), while the accuracy of Fastron RBF is lower (between 71–94%). Fastron RBF is only able to achieve 94% accuracy when there are 50 obstacles because almost all configurations are considered to be in $\mathcal{C}_{obs}$ at that point.

B. Motion Planning

We apply Fastron FK to a broad collection of the sampling-based motion planning algorithms implemented in OMPL [19]. We select RRT [20], RRT-Connect [21], SBL [22], RRT* [23], FMT* [24], and Informed-RRT* [25] to demonstrate each collision checking method's performance. RRT, its bidirectional variant RRT-Connect, and SBL are probabilistically complete motion planners [1] that terminate once a path between start and goal is found. SBL includes lazy collision checking, which only calls the collision check routine when absolutely necessary. The other planners are optimizing planners that attempt to determine the shortest collision-free path between start and goal. Rather than letting the optimizing planners run until timeout (which would cause all planning times to be equal), we stop these planners once an initial feasible path has been found. While these paths may not be close to optimal, we include the average lengths of these initial paths in the plot of results.

We use similar environments as used in Section IV-A, but the obstacle is sized such that the arm must always move around it; these environments are more challenging than in [5] because the volume of $\mathcal{C}_{free}$ is smaller with a larger obstacle. The start and goal configurations are selected to be on opposite sides of the workspace obstacle. Each collision checker is applied to each motion planner for 50 trials, where a trial consists of up to 30 random movements of the workspace obstacle, and the motion planner is repeated 10 times for each obstacle position. When using the approximate collision checkers, we verify and repair (if necessary) each plan using FCL by replanning with FCL on

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.

the invalid part of the plan. The resulting motion plan is thus guaranteed to be collision-free according to FCL.

The average motion planning timings are shown in Fig. 6. Even with the verify and repair steps, Fastron FK provides the fastest motion plans, providing up to 3 times faster collision-free motion plans compared to when using FCL. On the other hand, Fastron RBF often struggles to find a feasible path because it overestimates the $\mathcal{C}_{obs}$ region, causing its planning time to often be large or hit the time limit. Furthermore, the repair times for Fastron RBF are significantly larger than those for Fastron FK and GJK, illustrating that Fastron RBF often yields invalid paths.

V. Conclusion

In this letter we introduce the FK kernel and use it with the Fastron algorithm for proxy collision detection for manipulator arms. The FK kernel utilizes control points placed along a kinematic chain to better represent the relation between C-space and workspace. The FK kernel is positive definite, guaranteeing Fastron will provide a model with positive margin for a dataset in a finite number of iterations.

Compared to the previously-used RBF kernel, the FK kernel required 8 times fewer support points to represent the C-space model and allowed 2 times faster proxy collision detection. Prediction accuracy is significantly higher with the FK kernel (96%) than with the RBF kernel (74%). The FK kernel allowed proxy collision checking to be 9 times faster and motion planning to be up to 3 times faster than when using geometric methods.

Future work includes utilization of GPU parallelization for faster kernel evaluation, combining the effects of kernels applied in both configuration space and workspace, and applications of the FK kernel in other kernel-based methods.

References

[1] H. Choset *et al.*, *Princ. Robot Motion: Theory, Algorithms, Implementation*. Cambridge, MA, USA: The MIT Press, 2005.

[2] T. Lozano-Perez, "Spatial Planning : A configuration space approach," *IEEE Trans. Comput.*, vol. 32, no. 2, pp. 108–120, Feb. 1983.

[3] J. Pan and D. Manocha, "Efficient configuration space construction and optimization for motion planning," *Eng.*, vol. 1, no. 1, pp. 046–057, 2015. [Online]. Available: http://linkinghub.elsevier.com/retrieve/pii/S2095809916300443

[4] M. Elbanhawi and M. Simic, "Sampling-based robot motion planning: A review," *IEEE Access*, vol. 2, pp. 56–77, 2014.

[5] N. Das and M. Yip, "Learning-based proxy collision detection for robot motion planning applications," pp. 1–19, 2019, *arXiv:1902.08164v1* [cs].

[6] J. Pan, X. Zhang, and D. Manocha, "Efficient Penetration Depth Approximation using Active Learning," *ACM Trans. Graph.*, vol. 32, no. 6, 2013, Art. no. 191.

[7] J. Huh and D. D. Lee, "Learning high-dimensional Mixture Models for fast collision detection in Rapidly-Exploring Random Trees," in *Proc. - IEEE Int. Conf. Robot. Autom.*, 2016, pp. 63–69.

[8] J. Pan and D. Manocha, "Fast probabilistic collision checking for sampling-based motion planning using locality-sensitive hashing," *Int. J. Robot. Res.*, vol. 35, no. 12, pp. 1477–1496, 2016.

[9] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Proc. IEEE Int. Conf. Robot. Autom.*, 1985, pp. 500–505.

[10] D. Rakita, B. Mutlu, and M. Gleicher, "RelaxedIK: Real-time synthesis of accurate and feasible robot arm motion," in *Proc. Robot.: Sci. Syst.*, Jun. 2018.

[11] C. R. A. Rocha, C. P. Tonetto, and A. Dias, "Robotics and Computer-Integrated Manufacturing A comparison between the Denavit Hartenberg and the screw-based methods used in kinematic modeling of robot manipulators $," *Robot. Comput. Integr. Manuf.*, vol. 27, no. 4, pp. 723–728, 2011. [Online]. Available: http://dx.doi.org/10.1016/j.rcim.2010.12.009

[12] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: The MIT Press, 2006.

[13] T. Hofmann, B. Schölkopf, and A. J. Smola, "Kernel methods in machine learning," *Ann. Statist.*, vol. 36, no. 3, pp. 1171–1220, 2008.

[14] H. Q. Minh, P. Niyogi, and Y. Yao, "Mercer's Theorem, Feature Maps, and Smoothing," in *Proc. Int. Conf. Comput. Learn. Theory*, 2006, pp. 154–168.

[15] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[16] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, Apr. 1988.

[17] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *Proc. - IEEE Int. Conf. Robot. Autom.*, 2012, pp. 3859–3866.

[18] S. Chitta, I. Sucan, and S. Cousins, "Moveit!" *IEEE Robot. Autom. Mag.*, vol. 19, no. 1, pp. 18–19, Mar. 2012.

[19] I. A. Sucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, Dec. 2012.

[20] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," *Animal Genetics*, vol. 39, no. 5, pp. 561–563, 2008.

[21] J. J. Kuffner and S. M. Lavalle, "RRT-Connect : An Efficient Approach to Single-Query Path Planning," in *Proc. Int. Conf. Robot. Autom.*, Apr. 2000, pp. 995–1001.

[22] G. Sanchez and J.-C. Latombe, "A Single-Query Bi-Directional Probabilistic Roadmap," in *Proc. Tenth Int. Symp. Robot. Res.*, 2003, pp. 403–417.

[23] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 1–76, 2011.

[24] J. A. Starek *et al.*, "An Asymptotically-Optimal Sampling-Based Algorithm for Bi-directional Motion Planning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2015, pp. 2072–2078.

[25] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed RRT *: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2014, pp. 2997–3004.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:14:20 UTC from IEEE Xplore. Restrictions apply.